

// Parallel Reduction using OpenMP

```
#include <iostream>
#include <omp.h>      // OpenMP header
#include <climits>    // For INT_MAX, INT_MIN
using namespace std;

int main() {
    int arr[] = {3, 1, 7, 2, 9, 4, 6, 8, 5}; // Input array
    int n = 9;                               // Size of array
    // ===== DYNAMIC INPUT =====
    int n;
    cout << "Enter number of elements: ";
    cin >> n;
    int arr[n];
    cout << "Enter array elements: ";
    for(int i = 0; i < n; i++) {
        cin >> arr[i];
    }
    int minVal = INT_MAX; // Start with largest possible value- So any smaller number can replace it
    int maxVal = INT_MIN; // Start with smallest possible value
    long long sum = 0;    // Sum starts at 0- long long is used because sum may become large
    // OpenMP Parallel Reduction - Work is divided among multiple threads
    #pragma omp parallel for reduction(min:minVal) \
                        reduction(max:maxVal) \
                        reduction(+:sum)
    for (int i = 0; i < n; i++) {
        if (arr[i] < minVal) // Find minimum value
            minVal = arr[i];
        if (arr[i] > maxVal) // Find maximum value
            maxVal = arr[i];
        sum += arr[i];       // Calculate sum
    }
    // OpenMP combines all local thread results automatically
    // Calculate average
    double avg = (double)sum / n;
    cout << "Min: " << minVal << endl;    // Display results
    cout << "Max: " << maxVal << endl;
    cout << "Sum: " << sum << endl;
    cout << "Avg: " << avg << endl;
    return 0;
}

// Compile:
// g++ -fopenmp reduction.cpp -o reduction
// Run:
// ./reduction or reduction.exe
output:Min: 1 Max: 9 Sum: 45 Avg: 5
```

// Parallel BFS and DFS using OpenMP

```
#include <iostream>
#include <vector>
#include <queue>
#include <omp.h>          // OpenMP header for parallelism

using namespace std;

const int MAX = 100;      // Maximum number of nodes
vector<int> adj[MAX];      // Adjacency list for graph
bool visited[MAX];        // Array to track visited nodes

// ===== PARALLEL BFS =====
void bfs(int start, int n) {
    fill(visited, visited + n, false); // Reset visited array
    queue<int> q;                       // Queue used in BFS
    visited[start] = true;              // Mark starting node as visited
    q.push(start);                     // Push start node into queue
    cout << "BFS: ";
    while(!q.empty()) {
        int node = q.front();          // Get front node from queue
        q.pop();                       // Remove node from queue
        cout << node << " ";
        // Process neighbors in parallel
        #pragma omp parallel for
        for(int i = 0; i < adj[node].size(); i++) {
            int neighbor = adj[node][i]; // Get adjacent node
            if(!visited[neighbor]) {     // Check if neighbor is not visited
                // Critical section prevents race condition
                #pragma omp critical
                {
                    if(!visited[neighbor]) {
                        visited[neighbor] = true; // Mark neighbor visited
                        q.push(neighbor);        // Push into queue
                    }
                }
            }
        }
    }
    cout << endl;
}

// ===== DFS =====
void dfs(int node) {
    visited[node] = true;               // Mark current node visited
```

```

    cout << node << " ";
    for(int i = 0; i < adj[node].size(); i++) { // Visit all adjacent nodes
        int neighbor = adj[node][i];
        if(!visited[neighbor]) { // Visit only unvisited nodes
            dfs(neighbor); // Recursive DFS call
        }
    }
}

int main() {
    int n = 6; // Total number of nodes
    adj[0].push_back(1); // Create undirected graph
    adj[1].push_back(0);
    adj[0].push_back(2);
    adj[2].push_back(0);
    adj[1].push_back(3);
    adj[3].push_back(1);
    adj[1].push_back(4);
    adj[4].push_back(1);
    adj[2].push_back(5);
    adj[5].push_back(2);
    bfs(0, n); // Perform BFS traversal
    fill(visited, visited + n, false); // Reset visited array for DFS
    cout << "DFS: ";
    // Perform DFS traversal
    dfs(0);
    cout << endl;
    return 0;
}

/*
Compile:
g++ -fopenmp bfs_dfs.cpp -o bfs_dfs
Run: ./bfs_dfs OR bfs_dfs.exe
BFS: 0 1 2 3 4 5
DFS: 0 1 3 4 2 5
*/

```

```

[1] import numpy as np //Boston
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler # For normalizing data
import tensorflow as tf
from tensorflow import keras
# Load the dataset from CSV file
[2] df = pd.read_csv('1_boston_housing.csv')
print(df.head())
# Separate features (X) and target price (y)
[3] X = df.drop('MEDV', axis=1).values # All columns except price
y = df['MEDV'].values # 'medv' = median house value
# Split into training (80%) and testing (20%) sets
[4] X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)
# Normalize features — neural networks work better with scaled data
[5] scaler = StandardScaler()
X_train = scaler.fit_transform(X_train) # Fit on train, transform train
X_test = scaler.transform(X_test) # Only transform test (no fit!)
# Build the neural network model
[6] model = keras.Sequential([
    keras.layers.Dense(64, activation='relu', # First hidden layer, 64 neurons
        input_shape=(X_train.shape[1],)),
    keras.layers.Dense(32, activation='relu'), # Second hidden layer, 32 neurons
    keras.layers.Dense(1) # Output layer — 1 value (price)
])
# Compile model with loss function and optimizer
[7] model.compile(
    optimizer='adam', # Adam optimizer adjusts learning rate
    loss='mse', # MSE = Mean Squared Error (regression loss)
    metrics=['mae'] # MAE = Mean Absolute Error (readable metric)
)
# Train the model for 100 epochs
[8] history = model.fit(
    X_train, y_train,
    epochs=100, # 100 passes over the training data
    batch_size=32, # Process 32 samples at a time
    validation_split=0.1, # 10% of train data for validation
    verbose=1 # Print progress
)
# Evaluate model on test data
[9] mse, mae = model.evaluate(X_test, y_test, verbose=0)
print(f"Test MSE: {mse:.2f}")
print(f"Test MAE: {mae:.2f}") # Lower MAE = better predictions
# Predict on first 5 test samples

```

```
predictions = model.predict(X_test[:5])  
print("Predicted prices:", predictions.flatten())  
print("Actual prices:", y_test[:5])
```

IMDB Sentiment Classification using Deep Neural Network

```
[1] import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
import tensorflow as tf
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences
from tensorflow.keras import layers, models

[2] import pandas as pd
df = pd.read_csv(
    'IMDB_Dataset.csv',
    encoding='latin-1',
    engine='python',
    on_bad_lines='skip'
)
print(df.head())
print(df['sentiment'].value_counts())

[3] # Convert labels: 'positive' → 1, 'negative' → 0
le = LabelEncoder()
y = le.fit_transform(df['sentiment']) # Encodes text labels to numbers
texts = df['review'].values           # Raw text reviews

[4] # Tokenization — convert words to numbers
max_words = 10000                     # Only top 10,000 words
max_len = 200                         # Max words per review
tokenizer = Tokenizer(num_words=max_words)
tokenizer.fit_on_texts(texts)          # Build vocabulary from text
X = tokenizer.texts_to_sequences(texts) # Convert text to number sequences
X = pad_sequences(X, maxlen=max_len)   # Pad shorter reviews with 0s

[5] # Split data
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)

[6] # Build the neural network
model = models.Sequential([
    # Embedding: converts word indices into dense vectors
    layers.Embedding(max_words, 32, input_length=max_len),
    # Flatten the 2D embedding into 1D for Dense layers
    layers.GlobalAveragePooling1D(),
    layers.Dense(64, activation='relu'), # Hidden layer
    layers.Dropout(0.5),                 # Randomly drops 50% neurons (prevents overfitting)
    layers.Dense(1, activation='sigmoid') # Output: probability 0-1 (sigmoid for binary)
])

[7] # Compile model for binary classification
model.compile(
    optimizer='adam',
```

```

    loss='binary_crossentropy',      # Loss for binary (0/1) problems
    metrics=['accuracy']           # We care about % correct
)

[8] model.summary()                # Print model architecture
[9]
# Train the model
history = model.fit(
    X_train, y_train,
    epochs=5,                      # 5 passes (text training is slower)
    batch_size=64,
    validation_split=0.1,
    verbose=1
)
[10] # Evaluate on test data
loss, acc = model.evaluate(X_test, y_test, verbose=0)
print(f"Test Accuracy: {acc*100:.2f}%")
[11] # Predict sentiment for a new review
sample = ["This movie was absolutely brilliant and wonderful!"]
seq = tokenizer.texts_to_sequences(sample)
seq = pad_sequences(seq, maxlen=max_len)
pred = model.predict(seq)[0][0]
print("Positive" if pred > 0.5 else "Negative", f"({pred:.2f})")
[12] model.summary()              # Print model architecture

```

Fashion MNIST Classification using CNN

```
[1] import numpy as np
import pandas as pd
import tensorflow as tf
from tensorflow.keras import layers, models

[2] # Load data from CSV files
train_df = pd.read_csv('fashion-mnist_train.csv')
test_df = pd.read_csv('fashion-mnist_test.csv')

[3] # Separate labels (y) from pixel values (X)
y_train = train_df['label'].values      # Class 0-9
X_train = train_df.drop('label', axis=1).values
y_test = test_df['label'].values
X_test = test_df.drop('label', axis=1).values

[4] # Reshape flat 784-pixel rows into 28x28 images
X_train = X_train.reshape(-1, 28, 28, 1) # -1 = all samples, 1 = grayscale
X_test = X_test.reshape(-1, 28, 28, 1)

[5] # Normalize pixel values from 0-255 to 0-1
X_train = X_train / 255.0                # Divide all by 255
X_test = X_test / 255.0

[6] # Class names for the 10 clothing categories
class_names = ['T-shirt', 'Trouser', 'Pullover', 'Dress',
               'Coat', 'Sandal', 'Shirt', 'Sneaker',
               'Bag', 'Ankle boot']

[7] # Build the CNN model
model = models.Sequential([
    # Conv layer: 32 filters of size 3x3, detects features like edges
    layers.Conv2D(32, (3,3), activation='relu', input_shape=(28,28,1)),
    # MaxPooling: reduces image size by taking max in each 2x2 block
    layers.MaxPooling2D(2, 2),
    # Second conv layer: 64 filters, detects more complex features
    layers.Conv2D(64, (3,3), activation='relu'),
    layers.MaxPooling2D(2, 2),
    # Flatten 2D maps to 1D vector for Dense layers
    layers.Flatten(),
    layers.Dense(128, activation='relu'),    # Fully connected layer
    layers.Dropout(0.3),                    # Prevent overfitting
    # Output: 10 neurons (one per class), softmax gives probabilities
    layers.Dense(10, activation='softmax')
])

[8] # Compile model for multi-class classification
model.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy', # For integer labels (not one-hot)
    metrics=['accuracy']
)
```

```
[9] model.summary()                # Print layers and parameters
[10] # Train the model
model.fit(X_train, y_train,
          epochs=10,                # 10 passes through training data
          batch_size=64,
          validation_split=0.1,
          verbose=1)
[11] model.summary()
[12] # Evaluate on test data
test_loss, test_acc = model.evaluate(X_test, y_test, verbose=0)
print(f"Test Accuracy: {test_acc*100:.2f}%")
[13] # Predict class of first test image
pred = model.predict(X_test[:1])    # Get probability array
print("Predicted:", class_names[np.argmax(pred)]) # Pick highest prob class
print("Actual:", class_names[y_test[0]])
```